

# Xilinx 的 SerDes 接口介绍

因为摄像头输出的 LVDS 信号速率会达到 600Mbps,我们将不能够通过 FPGA 的 I/O 接口直接去读取这么高速率的信号。因此,需要使用 Xilinx FPGA 内的 SerDes 去实现高速数据的串并转换。

熊猫君的文章“Zynq 高速串行 CMOS 接口的设计与实现”,都已经说清楚了,大神~~

参考文档 ug953, ug471, 我们为了捕获 OV7251 摄像头 LVDS 的数据信号,将会使用的以下资源:

- IDELAYCTRL
- IDELAYE2
- ISERDESE2
- ODELAYE2 (用于模拟 LVDS 输出信号)
- OSERDESE2 (用于模拟 LVDS 输出信号)
- MMCME2\_ADV
- BUFG, BUFR, BUFIO, IBUFG, IBUFDS, OBUFDS

## 1. IDELAYCTRL

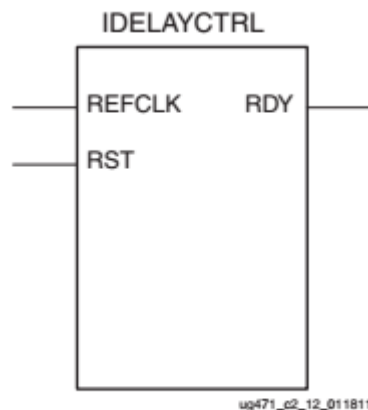


Figure 2-14: IDELAYCTRL Primitive

因 FPGA 的电压偏置、制造过程、电压、温度的不同,可能会对整个芯片的时序造成一些小的影响。IDELAYCTRL 可以通过一个较高频率的参考时钟 REFCLK 为 IDELAY 或 ODELAY 提供延时抽头,可选 0~31。参考时钟的频率可选 200MHz 或者 300MHz,它们的每个抽头的分辨率分别约是 78ps 和 52ps。我们的 LVDS 输入为 600Mbps,则选用 200MHz 需要 21taps, 300MHz 需要 32taps, 所以最终选择 200MHz 为参考时钟。

使用时还需要注意需要对 IDELAYCTRL 进行 LOC 约束,实现工具将 IDELAYCTRL 实例自动复制到整个器件,甚至复制到未使用延迟单元的时钟区域中。这样做资源占用率较高,在每个时钟区域内都要使用一个全局 时钟资源,并且使用布线资源较多,因此功耗较大。参考文档:输入输出延迟单元 IODELAY 简介

## 2. IDELAYE2

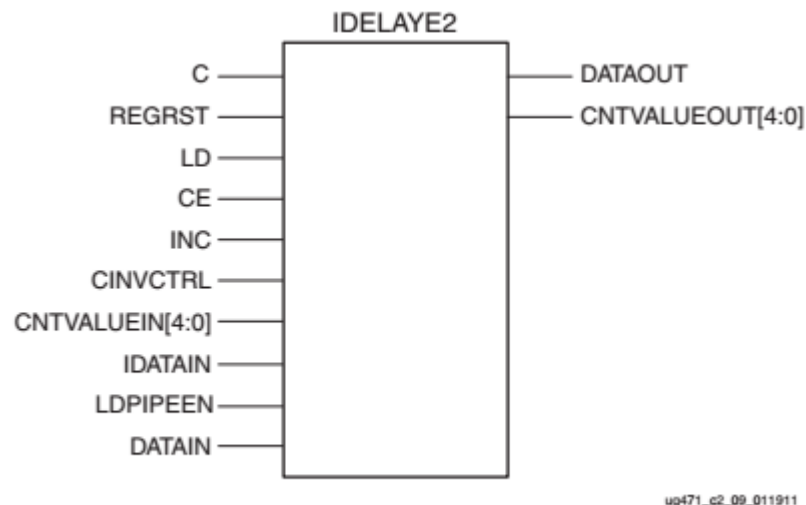


Figure 2-11: IDELAYE2 Primitive

IDELAYE2 逻辑是一个 31 抽头的循环延时补偿模块，对输入的信号进行指定分辨率的延时，从而保证从外部采集到的时钟和数据信号对应的建立保持时间在可接受的范围内。简单的说就是通过配置不同的 Tap 值，将 600MHz 的输入信号实现在 1.667ns 周期范围内进行相位调整。

为了调整时不出现数据的信号的断裂或者过分的变化，我们需要对 Tap 值每次只进行加一或者减一操作，不能一次调整的过大。参考 xapp585 进行动态相位补偿设计，章节：SDR Data Reception with Per-Bit Deskew。

## 3. ISERDESE2

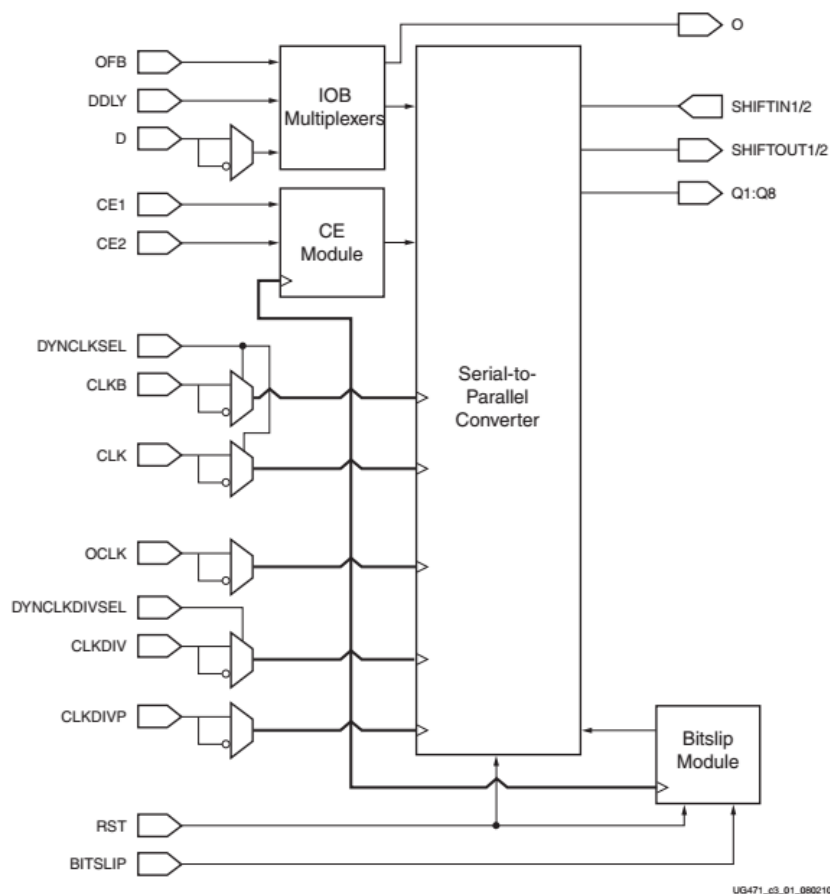


Figure 3-1: ISERDESE2 Block Diagram

ISERDESE2 就是我们的核心模块了，负责将串行数据转换为并行数据。  
 单个 ISERDESE2 的位宽只有 8bit，而我们的数据是 10bit，好在我们可以把两个 ISERDESE2 进行级联实现 10bit 串并转换。结构如下图，将 bit8 和 bit9 分别连为 Slave 的 Q3 和 Q4。

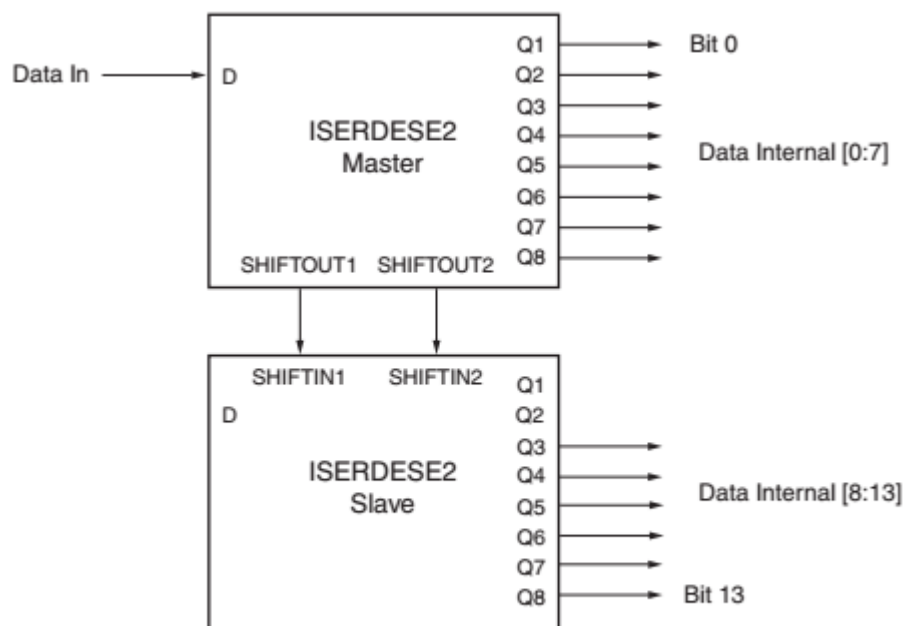


Figure 3-8: Block Diagram of ISERDESE2 Width Expansion

虽然前面通过 IDELAYE2 对数据的相位进行了调整，数据的建立保持时间都 OK，可以正常捕获串行数据了。但是，OV7251 只有一对差分时钟线和一对差分数据线，而没有额外的串行数据同步信号线，我们必须对串行数据进行字对齐（对于我们来说，像素数据是 10bit，所以是进行 10bit 对齐）。这就需要使用 ISERDESE2 中的 Bitslip 功能模块了。

Bitslip 功能是通过 BITSLLIP 输入信号来调整数据对齐的，BITSLLIP 信号的参考时钟是 CLKDIV。每发送一次 BITSLLIP 信号，对齐边沿就会按照下图的方式变化，SDR 和 DDR 模式的变化方式不同。

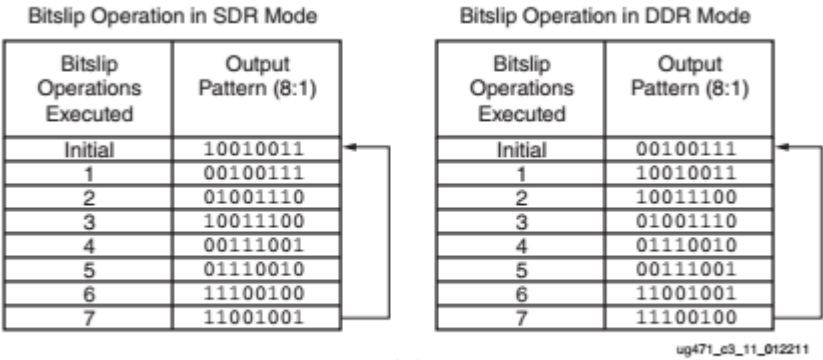
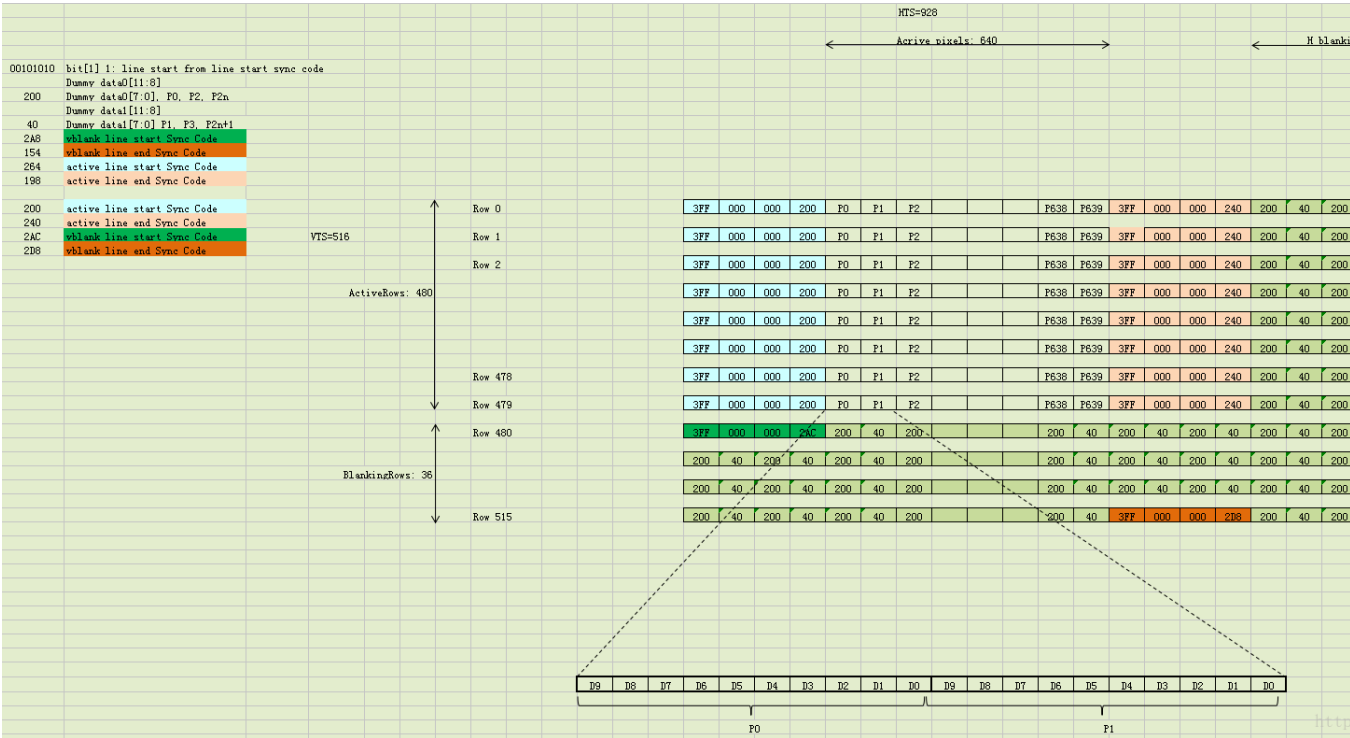


Figure 3-11: Bitslip Operation Examples

OV7251 发出的数据是 DDR 方式，所以是按照上图右边的方式进行变化：右移 1 位，左移 3 位……。而前面说到，既然没有同步信号线，我们就只能使用数据线中的同步码 Sync Code 去实现字对齐。OV7251 的帧格式如下图，我可以使用编码 3FF 000 000 2AC 来实现 BITSLLIP 调整。



我的处理方式和熊猫君的方式相同：图像开始传输后，在每一帧中都去搜索固定的同步码

“3FF 000 000 2AC”。如果在这一帧找到了同步码则表示同步完成，若没有找到同步码，则在下一帧开始前触发一次 **BITSLIP** 信号，然后继续查找。直到找到了同步码为止，或者超过了 20 次 **BITSLIP** 仍然没有找到，则代表同步失败。

**ODELAYE2** 和 **OSERDESE2** 相对来说没有那么复杂，就不介绍了。

#### 4. **MMCME2\_ADV**

时钟

#### 5. **BUF**

buf